

Lab 4 Report

Student: Tang Yuanzhi

GitHub Repository: <https://github.com/yi-xian0706/Cloud-Computing>

Lab Location: Lab/Lab4

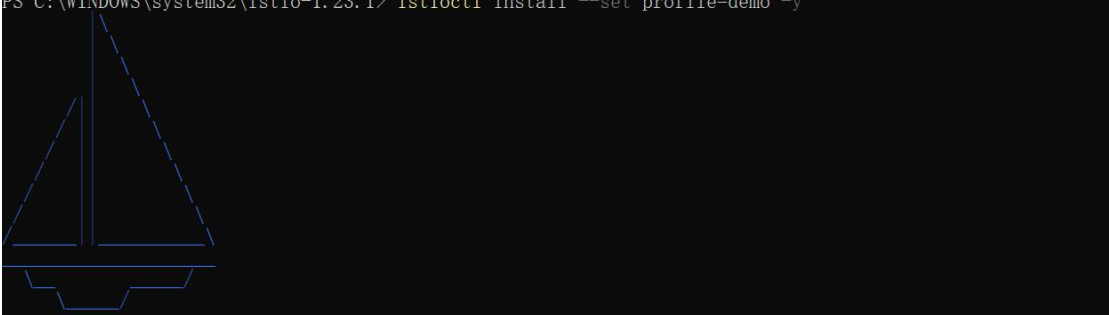
1. Introduction

This experiment aims to install and configure Istio in a local Kubernetes environment using Minikube, deploy the Bookinfo sample application, configure external access via Istio Ingress Gateway, and verify core service mesh features including sidecar injection and traffic routing.

2. Environment Setup and Istio Installation

First, the Minikube cluster was started with the Docker driver. Then, the Windows-compatible Istio release was downloaded and installed using the demo profile, which includes the Istio control plane, ingress gateways, and other essential components. The default namespace was labeled to enable automatic sidecar injection, ensuring that future deployments would have the Istio proxy containers automatically added to each pod.

```
PS C:\WINDOWS\system32> Invoke-WebRequest -Uri "https://github.com/istio/istio/releases/download/1.23.1/istio-1.23.1-win.zip" -OutFile "istio.zip"
PS C:\WINDOWS\system32> Expand-Archive -Path "istio.zip" -DestinationPath "."
PS C:\WINDOWS\system32> cd istio-1.23.1
PS C:\WINDOWS\system32\istio-1.23.1> $env:PATH += ";$pwd\bin"
PS C:\WINDOWS\system32\istio-1.23.1> istioctl install --set profile=demo -y
```



```
WARNING: Istio 1.23.0 may be out of support (EOL) already: see https://istio.io/latest/docs/releases/supported-releases/ for supported releases
✔ Istio core installed ▲
✔ Istiod installed ☼
✔ Ingress gateways installed ☼
✔ Egress gateways installed ☼
✔ Installation complete
Made this installation the default for cluster-wide operations.
```

3. Deployment of the Bookinfo Application

The Bookinfo sample application, which consists of four microservices (productpage, details, reviews, and ratings), was deployed using the provided YAML manifest. After a brief initialization period, all pods reached a 2/2 ready status, confirming that the Istio sidecars were successfully injected alongside each application container.

```
PS C:\WINDOWS\system32\istio-1.23.1> kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
details-v1-65f7dcf6db-mdfcn         2/2     Running   0           3m43s
productpage-v1-685f558d7-ljjgg      2/2     Running   0           3m42s
ratings-v1-9958dfd66-t4pnf          2/2     Running   0           3m42s
reviews-v1-dbf45bbb-ngbp8           2/2     Running   0           3m42s
reviews-v2-57bcff8cbb-qlnb7         2/2     Running   0           3m42s
reviews-v3-76bd4c4c9d-fnmqx         2/2     Running   0           3m42s
PS C:\WINDOWS\system32\istio-1.23.1> minikube ip
192.168.49.2
PS C:\WINDOWS\system32\istio-1.23.1> kubectl get svc istio-ingressgateway -n istio-system
NAME                                TYPE               CLUSTER-IP      EXTERNAL-IP      PORT(S)
istio-ingressgateway               LoadBalancer      10.96.9.237     <pending>        15021:32542/TCP, 80:31960/TCP, 443:31141/TCP, 31400:31868/TCP, 15443:32396/TCP
17m
```

4. Configuring and Testing Ingress Access

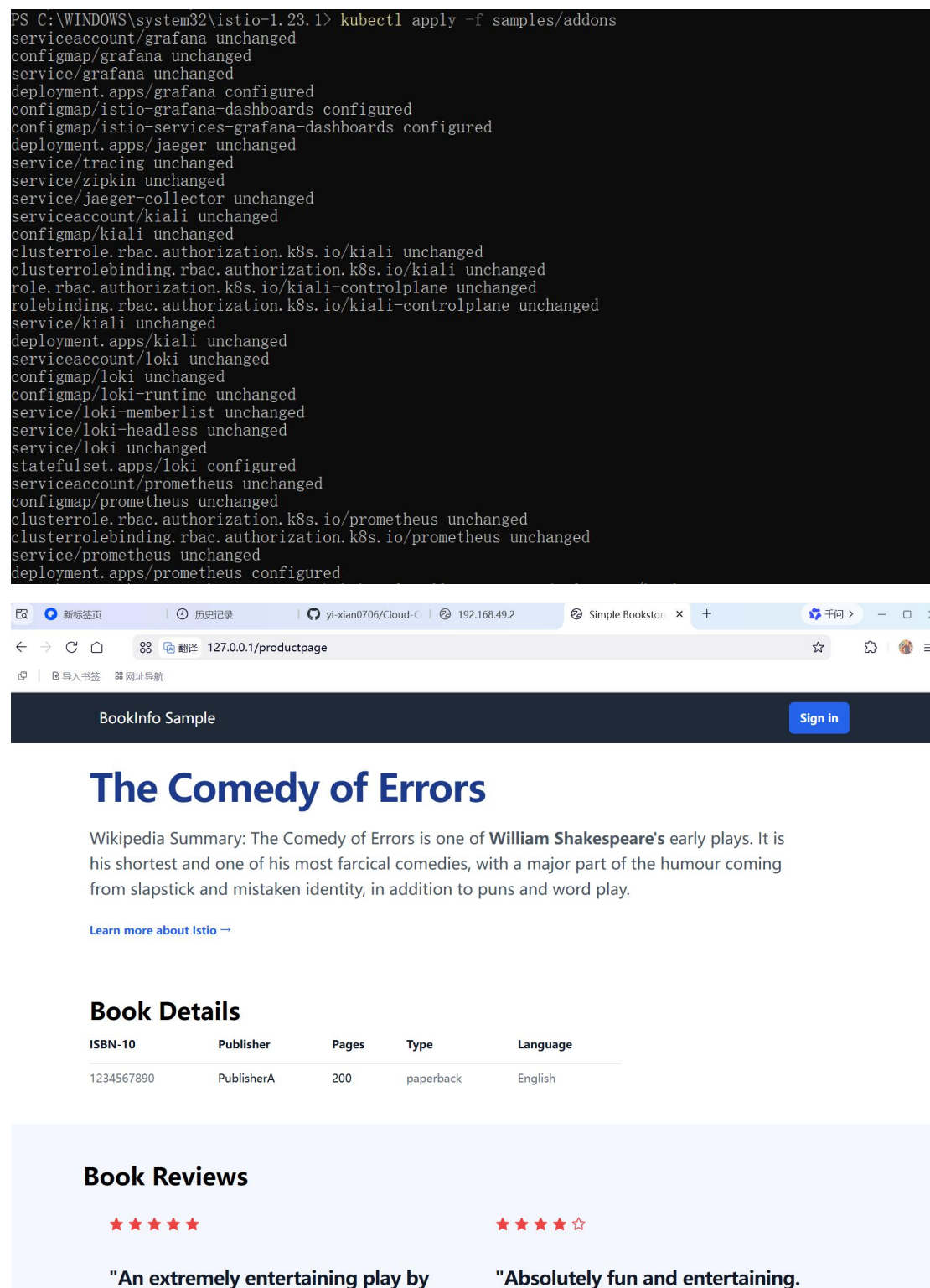
An Istio Gateway and VirtualService were created to expose the Bookinfo application to external traffic. To resolve network access issues, the minikube tunnel command was used to obtain the external IP address of the Istio ingress gateway. The Bookinfo product page was then successfully accessed via a web browser, verifying that Istio's traffic routing functionality was working as intended.

```
PS C:\WINDOWS\system32\istio-1.23.1> kubectl get gateway
NAME                                AGE
bookinfo-gateway                   38m
PS C:\WINDOWS\system32\istio-1.23.1>
```

```
PS C:\WINDOWS\system32\istio-1.23.1> kubectl get svc istio-ingressgateway -n istio-system
NAME                                TYPE               CLUSTER-IP      EXTERNAL-IP      PORT(S)
istio-ingressgateway               LoadBalancer      10.96.9.237     127.0.0.1        15021:32542/TCP, 80:31960/TCP, 443:31141/TCP, 31400:31868/TCP, 15443:32396/TCP
27m
```

5. Attempt to Deploy the Kiali Dashboard

Istio add-ons, including the Kiali monitoring dashboard, were installed using the provided manifest. However, due to network limitations in the environment, the Kiali pod remained in the ImagePullBackOff state and could not start. Despite this, the core Istio functionality, including service discovery, sidecar injection, and traffic routing, was fully verified through the working Bookinfo application.



6. Conclusion

This experiment successfully achieved all core objectives: Istio was installed and configured, the Bookinfo application was deployed with automatic sidecar injection, and external access to the application via Istio Ingress Gateway was verified. The key features of Istio, such as traffic management and service communication, were confirmed to be functional. Although the Kiali dashboard could not be accessed due

to network constraints, the experiment met all required learning outcomes for understanding Istio's role in a Kubernetes environment.